

Como leer el diagrama de arquitectura del Retail Recommender

1. Proposito del grafico

- Resume la arquitectura local actual: frontend ecommerce separado, backend FastAPI, recomendador local v2, artifact parquet y PostgreSQL para logs y metricas.
- Muestra tambien el pipeline offline que genera el artifact usado por la API.
- No representa un deployment cloud productivo: es una arquitectura local de demo MLOps operable y explicable.

2. Orden correcto para leerlo

1. Usuario / Browser

- Punto de entrada: el usuario interactua con la tienda desde el navegador.

2. Frontend

- App React, Vite y TypeScript. Muestra productos, carrito, recomendaciones y metricas. Corre separado en localhost:5173.

3. Backend API

- FastAPI en localhost:8000. Expone /recommend, /health, /model-info y /metrics/summary.

4. Recommender v2

- Servicio local que carga el modelo y aplica scoring ajustado por popularidad.

5. Model artifact + products.csv

- item_item_recommender_v2.parquet contiene asociaciones; products.csv permite devolver nombres de producto legibles.

6. PostgreSQL

- Guarda requests exitosos en recommendation_logs y alimenta metricas operativas.

7. Metrics / Operational checks

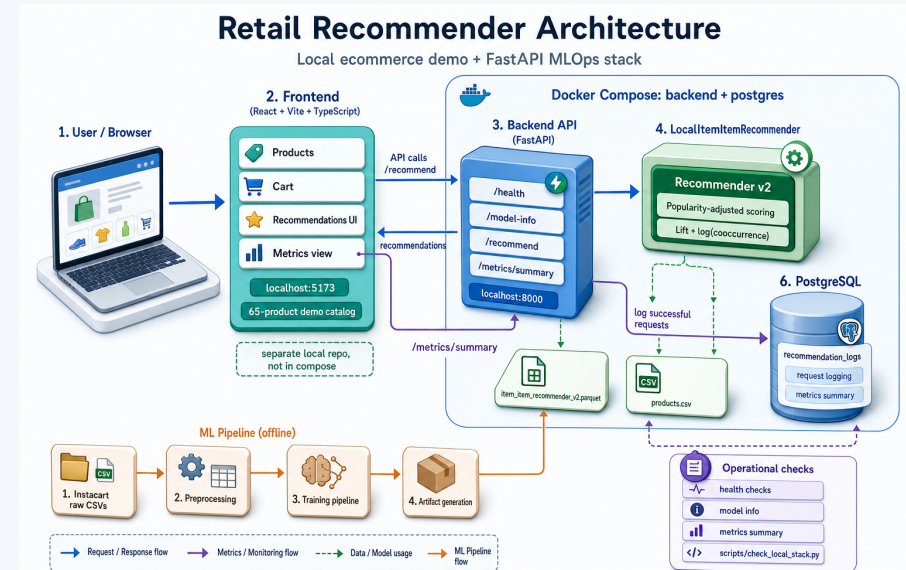
- Incluye resumen de metricas, health checks y scripts/check_local_stack.py.

8. ML Pipeline offline

- Raw CSVs -> preprocessing -> training -> artifact generation. Explica de donde sale el artifact servido por FastAPI.

Que muestra el diagrama

- Frontend ecommerce separado en localhost:5173.
- Backend FastAPI y PostgreSQL levantados con Docker Compose.
- Recommender v2 servido localmente desde un artifact parquet.
- Logs y metricas operativas basadas en recommendation_logs.



Referencia visual: architecture-isometric.png

Como leer el diagrama de arquitectura del Retail Recommender

3. Guion corto para explicarlo en 60-90 segundos

El sistema empieza en el navegador. El usuario interactúa con una tienda local construida en React, donde puede explorar productos, armar un carrito y ver recomendaciones. Ese frontend corre separado del backend, en localhost:5173. Cuando el usuario pide recomendaciones, la app llama al endpoint /recommend del backend FastAPI, que corre localmente en localhost:8000. FastAPI delega la inferencia al LocalItemItemRecommender, un servicio local que carga el artifact item_item_recommender_v2.parquet y usa products.csv para enriquecer los resultados con nombres y metadata. El modelo v2 aplica un scoring ajustado por popularidad, de modo que no se limita a recomendar solamente los productos mas frecuentes. Luego el backend responde al frontend con productos recomendados para el carrito. Además, cada request exitoso se registra en PostgreSQL, en la tabla recommendation_logs. Esos logs permiten construir metricas operativas como volumen, latencia promedio y cantidad promedio de recomendaciones. Finalmente, el bloque inferior muestra que el artifact no aparece magicamente: se genera offline a partir de los CSV de Instacart, pasando por preprocessing, training y generacion del parquet. Docker Compose, por ahora, levanta solo backend y PostgreSQL; el frontend sigue siendo una app local separada.

4. Mensaje principal para la audiencia

- El proyecto separa claramente frontend, backend y ML artifact.
- El modelo se sirve localmente via FastAPI, sin depender de cloud.
- El sistema ya incluye trazabilidad basica con logs y metricas.

5. Que no esta incluido todavia

No implementado todavia:

- auth
- cloud deployment
- CI/CD
- Kubernetes
- Vertex AI
- frontend dentro de Docker Compose

Nota para presentacion

Usa el diagrama como una historia de flujo: usuario -> frontend -> API -> recommender -> artifact/catalog -> respuesta -> logs/metricas. El pipeline offline se explica al final como origen del modelo.